CredShields

Smart Contract Audit

February 12, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Ploutos between February 9, 2026, and February 12, 2026. A retest was performed on February 12, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli(Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Ploutos

Table of Contents

Table of Contents **2**
1. Executive Summary ——————————- **3**
State of Security 4
2. The Methodology ——————————- **5**
2.1 Preparation Phase 5
2.1.1 Scope 5
2.1.2 Documentation 5
2.1.3 Audit Goals 6
2.2 Retesting Phase 6
2.3 Vulnerability classification and severity 6
2.4 CredShields staff 8
3. Findings Summary ——————————- **9**
3.1 Findings Overview 9
3.1.1 Vulnerability Summary 9
5. Bug Reports —————————— **10**
Bug ID #L001 [Acknowledged] 10
Addresses provider inconsistencies 10
Bug ID #L002 [Acknowledged] 11
Incorrect aToken Allowance Handling Due to Missing Scaling 11
Bug ID #L003 [Acknowledged] 12
Incorrect fees might be used for the bridge 12
Bug ID #L004 [Acknowledged] 13
Delegatecall Failure Not Properly Handled in Proxy Initialization 13
Bug ID #L005 [Acknowledged] 14
Potential bridge failure 14
Bug ID #I001 [Acknowledged] 15
Event MintUnbacked Field user is Always Set to Bridge 15
Bug ID #I002 [Acknowledged] 16
Configurator Decimals Are Parameters Rather than Fetched 16
6. The Disclosure ——————————- **17**

1. Executive Summary ----------

Ploutos engaged CredShields to perform a smart contract audit from February 9, 2026, to February 12, 2026. During this timeframe, 7 vulnerabilities were identified. **A retest was performed on February 12, 2026, and all the bugs have been acknowledged.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Ploutos" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Lending Deploy Contracts	0	0	0	5	2	0	**7**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Lending Deploy Contract's scope during the testing window while abiding by the policies set forth by Ploutos's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Ploutos's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Ploutos can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Ploutos can future-proof its security posture and protect its assets.

2. The Methodology ----------

Ploutos engaged CredShields to perform the Lending Deploy Contract's audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from February 9, 2026, to February 12, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Reviewed Scope: https://github.com/ploutusFi/lending-deploy/blob/c879d27851622dfdedf5979ae63526df59d11f76/README.md

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Ploutos is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat

agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities

can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 7 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Addresses provider inconsistencies	Low	Inconsistent Accounting (SCWE-010)	Acknowledged
Incorrect aToken Allowance Handling Due to Missing Scaling	Low	Inconsistent Accounting (SCWE-010)	Acknowledged
Incorrect fees might be used for the bridge	Low	Business Logic (SC03-LogicErrors)	Acknowledged
Delegatecall Failure Not Properly Handled in Proxy Initialization	Low	Improper Error Handling	Acknowledged
Potential bridge failure	Low	Business Logic (SC03-LogicErrors)	Acknowledged
Event MintUnbacked Field user is Always Set to Bridge	Informational	Event Design Issue	Acknowledged
Configurator Decimals Are Parameters Rather than Fetched	Informational	Validation Issue	Acknowledged

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #L001 [Acknowledged]

Addresses provider inconsistencies

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Low

Description
The unregisterAddressesProvider function of the PoolAddressesProviderRegistrycontract is doing a reset of the _addressesProviders mapping leaving the _addressesProvidersList array unmodified. For this reason, the getAddressesProvidersList function will iterate over old values that might have been deleted, making it very inefficient. Moreover, the documentation stated that 0 is used to represent the mainnet Pluotos market, but when unregistering an address provider, 0 value is used to deactivate that provider, conflicting with the previous definition.

Affected Code
- PoolAddressesProviderRegistry::#L52

Impacts
The inconsistent state between the _addressesProviders mapping and the _addressesProvidersList array can lead to stale or invalid entries being returned by getAddressesProvidersList, increasing gas costs and reducing efficiency over time.

Remediation
Consider reducing the array size by swapping the last element with the one that is going to be deleted and popping the last element from the array to correctly update the array size and content. This will change the order of the elements in the array but will definitely be more storage efficient for both saving and retrieving those values. Lastly, consider being consistent in the indexes used and their meanings, avoiding using the zero value as a meaningful representation of a valid state.

Retest
-

Bug ID #L002 [Acknowledged]

Incorrect aToken Allowance Handling Due to Missing Scaling

Vulnerability Type
Inconsistent Accounting (SCWE-010)

Severity
Low

Description
The AToken implements a scaled balance model where user balances are internally stored in scaled form and converted to actual values using the reserve liquidity index. Functions such as balanceOf, totalSupply, and transfers correctly apply scaling. However, the approval logic inherited from IncentivizedERC20 does not apply scaling. Both approve and permit store the raw value without converting it into scaled units.

This creates an inconsistency because balances are index-adjusted while allowances are not. A user approving an amount based on their visible balance may unintentionally grant a spender more effective spending power than intended. This breaks internal accounting consistency within the scaled token design.

Affected Code
- AToken.sol::#L170-#L193

Impacts
Allowance accounting becomes inconsistent with the scaled balance model, potentially leading to unintended over-spending and integration inconsistencies in DeFi integrations.

Remediation
Refactor approve and permit to store allowances in scaled units using the current liquidity index, ensuring alignment between balance and allowance accounting.

Retest
-

Bug ID #L003 [Acknowledged]

Incorrect fees might be used for the bridge

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
The bridge logic has two separate functions to mintUnbacked and backUnbacked. Moreover protocol fee is charged in backUnbacked calculated with the current protocolFeeBps. However, since the protocolFeeBps might change in between mintUnbacked and backUnbacked, it is not always suitable to use the latest protocolFeeBps as it might not be what the user has agreed on when calling the mintUnbacked function.

Affected Code
- BridgeLogic::#L116

Impacts
If protocolFeeBps changes between mintUnbacked and backUnbacked, users may be charged a different fee than the one implicitly assumed at the time of minting.

Remediation
Considering saving the protocolFeeBps during mintUnbacked and use the saved value to charge user instead of the latest protocolFeeBps.

Retest
-

Bug ID #L004 [Acknowledged]

Delegatecall Failure Not Properly Handled in Proxy Initialization

Vulnerability Type
Improper Error Handling

Severity
Low

Description

The initialize function in InitializableAdminUpgradeabilityProxy calls:

```
InitializableUpgradeabilityProxy.initialize(logic, data);
```

Inside this parent function, a delegatecall is executed to the logic contract with the provided data. If this delegatecall fails, the revert reason from the implementation may not be properly returned or bubbled up by the proxy. This makes debugging initialization failures difficult and reduces transparency during deployment or upgrades.
The same issue also exists in similar upgrade functions like upgradeToAndCall, where delegatecall errors are not properly handled.

Affected Code

- InitializableAdminUpgradeabilityProxy.sol::#L27

Impacts
If the implementation initialization fails, the proxy may not provide a clear revert reason. This makes it harder to detect configuration errors or logic issues during contract deployment or upgrade.

Remediation
Verify the result of the delegatecall. If successful, return the returndata. If failed, bubble up the original revert reason or revert with a clear custom error message.

Retest
-

Bug ID #L005 [Acknowledged]

Potential bridge failure

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
The PoolConfigurator contract allows pool or risk admins to set the unbackedMintCap parameter to any value, including 0. However, the BridgeLogic contract relies on unbackedMintCap inside the mintUnbacked function, where it requires the value to be positive. If unbackedMintCap is set to 0, calls to mintUnbacked will revert, potentially breaking bridge functionality.

Additionally, the explicit check that unbackedMintCap must be positive is redundant, since another condition in the same require statement enforces that:

```
unbackedMintCap > unbacked / (10 ** reserveDecimals)
```

and unbacked is always non-negative.

Affected Code
- BridgeLogic.sol::#L68

Impacts
Setting unbackedMintCap to 0 can cause bridge minting operations to revert, leading to a denial of service for bridging functionality and inconsistent protocol behavior.

Remediation
Enforce unbackedMintCap to be non-zero at the configuration level or update BridgeLogic to correctly handle the zero case. Additionally, refactor the require statement to remove redundant checks and improve clarity.

Retest
-

Bug ID #I001 [Acknowledged]

Event MintUnbacked Field user is Always Set to Bridge

Vulnerability Type
Event Design Issue

Severity
Informational

Description
The event MintUnbacked contains the field user .This field is set in BridgeLogic.executeMintUnbacked() to be msg.sender. However since this function can only be called by the bridge, msg.sender will always be the bridge.

Affected Code
- BridgeLogic.sol::#L102

Impacts
Since the user field in the MintUnbacked event is always set to msg.sender and the function can only be called by the bridge, the emitted value will always represent the bridge rather than the actual end user.

Remediation
Consider changing the field name to bridge rather than user . Alternatively, consider adding user as a parameter to the function mintUnbacked() which may be passed from the bridge and thus emitted correctly in the event.

Retest
-

Bug ID #I002 [Acknowledged]

Configurator Decimals Are Parameters Rather than Fetched

Vulnerability Type
Validation Issue

Severity
Informational

Description
Setting up reserves involves initialising required contracts; AToken , StabledDebtToken and VariableDebtToken . Each of these tokens are ERC20 derivatives and require decimals to be passed as an argument.
The value of the variable decimals is passed as a parameter to PoolConfigurator.initReserves() . There is the assumption this value should match the decimals of the underlying asset. However there are no checks to ensure this assumption holds.

Affected Code
- ConfiguratorLogic.sol

Impacts
The impact would be a misconfiguration of the deployment which could potentially go undetected until users are over or under rewarded for their transactions.

Remediation
This assumption can be enforced by instead fetching the decimals() function.

Retest
-

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

Audited by CRED SHIELDS

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com